

Optimization for Machine Learning Handout

Industrial Engineering Program

Activity 1: Binary Classification

1. Draw a series of stars on the provided sticker cards (create 5–10 cards) – see Figure 1.
 2. Place each card, one at a time, on top of the Arduino camera mounted on the plastic punched card – see Figure 2.
- Pay attention to the placement of Arduino in the plastic box. The camera is placed in the center of the box as shown with the red arrow.
3. Run the `reading_image.py` script and press Enter to capture each image before moving to the next card – see Figure 3.
 4. Next, draw a series of circles on another set of sticker cards (again, create 5–10 cards).
 5. Update the “Label” in the script to ‘0’ to indicate circles.

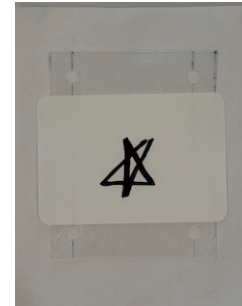


Figure 1: Example of a star drawing

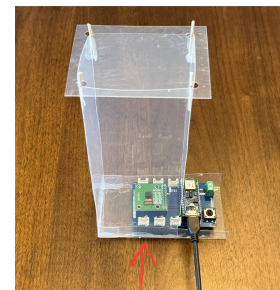


Figure 2: Arduino camera setup

```
1 import serial
2 import numpy as np
3 import os
4
5 # --- Parameters ---
6 PORT = 'COM7'
7 BAUD = 115200
8 WIDTH, HEIGHT = 376, 144
9 CROP_W, CROP_H = 160, 140
10 CROP_X, CROP_Y = (WIDTH - CROP_W) // 2, (HEIGHT - CROP_H) // 2
11 DOWNSAMPLED_SIZE = 28
12 BLOCK_W = CROP_W // DOWNSAMPLED_SIZE
13 BLOCK_H = CROP_H // DOWNSAMPLED_SIZE
14 NUM_IMAGES = 1 # how many images to collect
15 LABEL = 0 # 1 for capturing stars...change to 0 if capturing circles
16
17 # --- Output CSVs ---
18 image_csv_path = "images.csv"
19 label_csv_path = "labels.csv"
```

Figure 3: `reading_image` python code

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Load Data
5
6 # Normalized pixels
7 X = np.loadtxt("images.csv", delimiter=",", dtype=np.float32)
8 y = np.loadtxt("labels.csv", delimiter=",", dtype=np.float32).reshape(-1, 1)
9
10 # Convert labels from {0, 1} to {-1, +1}
11 y = 2 * y - 1
12
13 n_samples, n_features = X.shape
14 theta = np.zeros((n_features, 1)) # No bias term
```

Figure 4: `training_binary` python code

```
1 import serial
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # --- Parameters ---
6 PORT = 'COM7'
7 BAUD = 115200
8 WIDTH, HEIGHT = 376, 144
9 CROP_W, CROP_H = 160, 140
10 CROP_X, CROP_Y = (WIDTH - CROP_W) // 2, (HEIGHT - CROP_H) // 2
11 DOWNSAMPLED_SIZE = 28
12 BLOCK_W = CROP_W // DOWNSAMPLED_SIZE
13 BLOCK_H = CROP_H // DOWNSAMPLED_SIZE
14
15 # Load trained weights (theta) ---
16 theta = np.load("theta.npy") # shape: (400,)
17 theta = theta.flatten()
18 assert theta.shape == (400,), "Expected theta to be a (400,) vector"
19
20 # --- Sigmoid function ---
21 def sigmoid(z):
22     return 1 / (1 + np.exp(-z))
```

Figure 5: `testing_binary` python code

Questions: Which step-size lead to a smaller loss? What is your training accuracy?

9. Borrow a few cards from your neighbor and test your trained model. Place each card on top of the Arduino camera and run the `testing_binary.py` script to evaluate performance—see Figure 5.

Question: How many did you classify correctly?

Activity 2: Handwritten Digit Recognition

1. Open and run the file `training_MNIST.py`. It will output the file `mnist_model_quant.h` – See Figure 6.
2. Open the file `handwritten_digit_recognition.ino` in the Arduino IDE. Upload the file to the Arduino – See Figure 7.
3. Draw a series of digits (0–9) on the provided sticker cards (create 5–10 cards). Each card should have a **single clear digit** centered and black on a white background – See Figure 8.
4. Place each card, one at a time, on top of the Arduino camera mounted on the plastic punched card – see Figure 2.
5. In the Arduino Serial Monitor, press the onboard button to capture and classify the digit – see Figure 9.

To capture an image, first press the reset button, then press the capture button after a few second. Read the predicted digit either by counting the number of blinks on the device or read from the serial.

6. The model will return a predicted digit from 0–9.

Question: Which digits were classified correctly? Which ones were incorrect? Try showing poorly written digits. Does the model still work well?

7. Try editing the Python training script to change the neural network model. Start with a single layer with 100 neurons.

You can have at most three dense layers and a total of 800 neurons. It's a tiny machine learning!

Question: What happens if you increase the first layer from 100 to 700? How does it affect accuracy?

Question: Given the test collection, count the number of correct predictions!

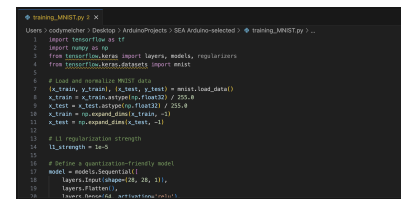


Figure 6: `training_MNIST.py` code

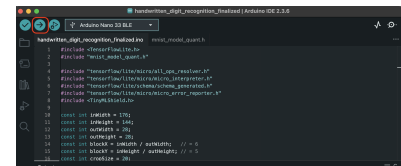


Figure 7: `handwritten_digit_recognition.ino` code

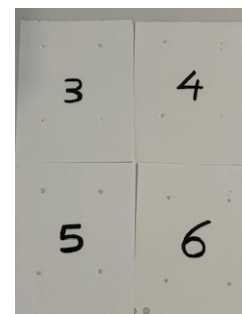


Figure 8: Example of digit drawings

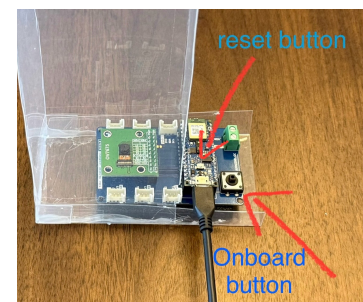


Figure 9: The reset and onboard buttons on the device

Neural Networks Training: Layers and Neurons

- **Input Layer:** Where the raw image enters the model. This layer flattens the image into a series of numbers.
- **Hidden Layers:** These layers learn patterns in the image (like curves, edges, or shapes). Each layer has a set of “neurons” (math units) that help learn those patterns.
- **Output Layer:** Makes the final prediction for the image (for example, which digit 0–9 the model believes it sees).

Layers vs Neurons:

- Increasing neurons gives a layer more power to learn a feature. More layers means the model can learn more complicated features.
- However, too many neurons or layers will be difficult to train and can lead your model to learn the specific example you have fed it instead of learning the general task; this is called overfitting.
- In contrast, too few neurons or layers will leave the model unable to sufficiently analyze enough patterns or features to learn the task.

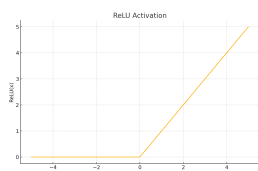
Your limit: Use up to 900 total neurons and 4 hidden layers, each layer not exceeding 750 neurons. Try splitting them across layers:

- One hidden layer: 100
- Three hidden layers: 150 + 100 + 50
- Four hidden layers: 400 + 200 + 100 + 50

Try different combinations of neurons and hidden layers to see what improves classification accuracy.

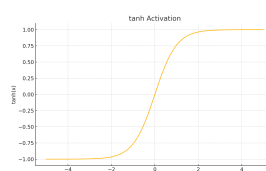
Activation Functions

ReLU



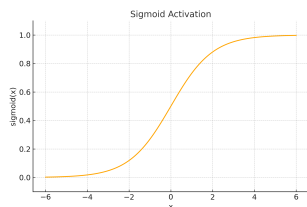
Keeps positive values, sets negatives to 0.

Tanh



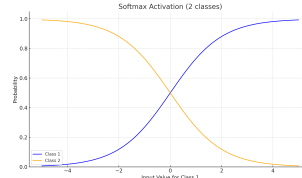
Smooth curve from -1 to 1.

Sigmoid



Smooth curve from 0 to 1.

Softmax



Converts outputs into classification probabilities.

- **ReLU:** Fast and simple, but can sometimes collapse around 0.
- **Tanh:** Useful if your data is around 0 or if you have shallow networks. Can learn slowly for training data not near 0.
- **Sigmoid:** Smooth curve for binary outputs. Good for yes/no decisions, but can slow down if output is near 0 or 1.
- **Softmax:** Use in the final layer when your model picks a class based on probability (like digit 0–9). Don't change the final layer in the given code from being softmax!